MediCore · Plan de trabajo por entregas

Cada entrega es pequeña, termina compilando, con pruebas verdes y la matriz de trazabilidad actualizada. Un flujo a la vez, comparado contra el diseño de referencia.

#	Entrega	Contenido	Estado
0	Preparación	Solución, API y web iniciales, CLAUDE.md, README, plan y trazabilidad, prueba de humo	**Hecha**
1	Base técnica	Capas del monolito modular (dominio/aplicación/infraestructura por módulo), EF Core + PostgreSQL con migraciones, `docker-compose` (Postgres, Redis, RabbitMQ), `.env.example`, configuración y secretos locales (user-secrets), manejo global de errores, Swagger/OpenAPI	**Hecha** (PR #1 fusionado)
2	Identidad y acceso	Usuarios, roles, login web (cookie) y API (JWT corto + refresh), sesiones en Redis, políticas por rol y por recurso, auditoría base, WEB-01, MOB-01 (API), usuarios y roles (WEB-10)	**Hecha** (PR #2 fusionado)
3	Pacientes y médicos	CRUD, búsqueda, `GET /api/patients/{id}`, `GET /api/doctors`, WEB-05…08, auditoría de accesos a datos clínicos	**Hecha** (PR #3 fusionado)
4	Agenda y citas	Disponibilidad (WEB-09), `GET /api/doctors/{id}/availability`, `POST /api/appointments`, cancelación, **anti doble reserva** (restricción única en BD + concurrencia optimista/transacción), cache Redis de disponibilidad con invalidación, WEB-02/03/04/11	**Hecha** (incluida en main con el PR #4)
5	Consultas y recetas	Expediente, consultas, recetas, `GET /api/prescriptions/{id}`, WEB-12…15, autorización por recurso	**Hecha** (PR #4 fusionado en main; incluye la entrega 4 y el expediente compartido)
6	Laboratorio y notificaciones asíncronas	`POST /api/labs/results` y consulta/listado, WEB-16…18, autorización por rol y por recurso, **RabbitMQ real**: evento `LabResultReady` con outbox transaccional, consumidor idempotente con reintentos y DLQ, API de notificaciones del paciente	Implementada en la rama `feature/entrega-6-laboratorio-notificaciones` (pendiente de PR y fusión)
7	Mensajería y notificaciones (resto)	Generalizar lo hecho en la entrega 6 (outbox, publicador, consumidor con reintentos, DLQ e idempotencia) a los eventos restantes: `AppointmentCreated`, `AppointmentCancelled`, `PrescriptionCreated` y `PatientNotification`; reintento de notificación	**Hecha**
8	Web del personal completa	Pulido contra el diseño, accesibilidad, responsive, español	**Hecha**: 116/116 en Cypress, 14 pantallas con axe
9	App móvil (trabajada como "entrega 7" de la numeración de sesiones)	.NET MAUI Android (MOB-01…12) sobre la API real, flujos Maestro y documentación de conexión con Docker	Implementada en la rama `feature/entrega-7-app-movil`; compilada a APK de desarrollo y **probada en emulador (Maestro 8/8)**; pendiente de PR y fusión
10	Pruebas	Unitarias, integración (BD/Redis/RabbitMQ con Testcontainers), API (Insomnia), concurrencia de agenda, Cypress, Maestro	**Hecha**
11	DevSecOps	Jenkinsfile (18 etapas), SonarQube, Dependency-Check, npm audit, Fortify, ZAP, Dockerfiles	**Hecha**: 18 etapas y bloqueo demostrado. Fortify sin licencia

#	Entrega	Contenido	Estado
12	Cloud (preparación = "entrega 8" de la numeración de sesiones)	Terraform (Azure), Ansible, aprovisionar/modificar/destruir, ambiente QA	Preparado y verificado en local. Azure es el último paso
13	Rendimiento y calidad	k6/JMeter con umbrales previos, Lighthouse/axe, prueba de usabilidad	k6 y axe hechos. Usabilidad pendiente de aplicar
14	Cierre	Plan de pruebas (20 secciones), matriz de riesgos, casos/evidencias, defectos, informe GO / NO-GO, presentación	**Hecha**: propuesta NO-GO para producción, GO condicionado para QA

Decisión aceptada (entregas 3–4): el médico accede a pacientes con quienes tiene una cita no cancelada **o una consulta registrada**; la regla se valida en la API (`PatientAuthorizationHandler`). Cancelar una cita solo retira el acceso si no queda otra relación válida (otra cita con ese médico o una consulta). La parte de consultas se implementó y se prueba en la entrega 5 (la prueba que estaba omitida ya está activa).

Pendientes (ver `13-informe-final.md`): usabilidad con personas, Fortify y despliegue en Azure.

Decisión aceptada (entrega 5, ajuste): **expediente compartido entre médicos autorizados.** Un médico con relación de atención válida lee las consultas y recetas que registraron otros médicos; nadie modifica registros de otro profesional; sin relación válida no hay acceso; los demás roles mantienen sus restricciones. Detalle en `CLAUDE.md` y en `04-registro-de-hallazgos.md`.

Decisión aceptada (entrega 6): el **evento** `LabResultReady` y la infraestructura de mensajería (outbox, publicador, consumidor con reintentos, DLQ e idempotencia) se adelantaron a la entrega 6 porque el flujo de laboratorio los necesita. La entrega 7 conserva el resto: los eventos `AppointmentCreated`, `AppointmentCancelled`, `PrescriptionCreated` y `PatientNotification`, y la generalización del consumidor.

Decisión aceptada (entrega 8): el proceso de mensajería se separa en el proyecto `MediCore.Worker` (mismo código de Infrastructure) para desplegarlo como contenedor; en desarrollo sigue dentro de la API. **Siguen pendientes y NO están hechos:** Jenkins (Jenkinsfile de 18 etapas), SonarQube, Dependency-Check/npm audit/Fortify, OWASP ZAP, Insomnia, k6/JMeter, ampliar Cypress y los entregables de QA de la entrega 14.

Dependencias clave: 2 antes de 3–6; 4 y 6 usan los eventos de 7 (se publican vía outbox desde 4 y 6 y se consumen en 7); 9 requiere API estable (≥ entrega 6).

Los entregables de documentación (plan de pruebas, riesgos, casos) se van construyendo en cada entrega y se consolidan en la 14.